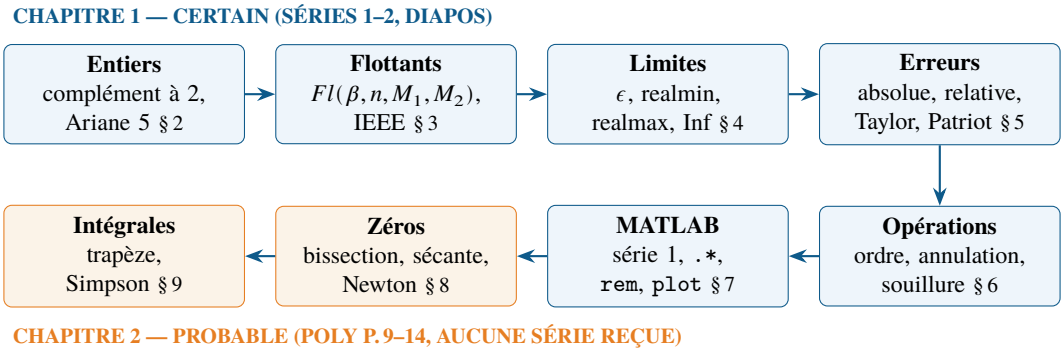
Mathématiques numériques — Guide d'apprentissage TE1

Ch. 1 nombres machine et erreurs · MATLAB (série 1) · Ch. 2 zéros et intégrales

Notations du polycopié 2026-27 et des diapositives de R. Baltensperger ; exemples des séries 1 et 2 et de leurs solutions officielles.
Compléments signalés comme tels : proposés, non officiels.

Chaque nombre sort de `scripts/guide_te1_calculs.py` (247 contrôles) ; les sorties MATLAB citées ont été relevées sous R2025a avec `scripts/guide_te1_controle.m`. **Portée du TE non annoncée par écrit : chapitre 1 certain, chapitre 2 probable — à confirmer en cours.**

Carte du TE : ce qui s'enchaîne avec quoi



Comment utiliser ce guide

Chaque notion suit le même ordre : la **définition du poly ou des diapos**, un **exemple chiffré** (source entre crochets) et le **piège**. Deux fils rouges : la **machine décimale à quelques chiffres** des diapositives (on arrondit après *chaque* opération) et la fonction du poly $f(x) = 3x + \sin x - e^x$ (§ 8). La veille : la **méthode** (§ 10), le **mémo des pièges** (§ 11) et l'**auto-test** (§ 12).

L'idée directrice en une phrase

L'ordinateur ne connaît qu'un *ensemble fini* de nombres : tout réel est remplacé par un voisin, avec une erreur relative d'au plus ϵ. Une erreur aussi petite ne devient grave que si on la *multiplie* (Patriot), si on *soustrait* deux nombres voisins (annulation) ou si on sort du domaine (Ariane 5, overflow). Les méthodes du chapitre 2 produisent elles aussi une approximation : il faut savoir *borner* leur erreur.

1. Le TE1 : règles du jeu

Points	Critère du manuel (exercice à 2 points)
2	tout est juste, valeurs numériques et méthode
1,5	presque tout juste (petite faute sans gravité)
1	en partie correct, plusieurs erreurs d'importance
0,5	presque tout faux, au moins une idée correcte
0	il manque une grande quantité d'informations

Vendredi 06.11.2026, 8 h 15–9 h 15, D20-12 (60 min, obligatoire ; moyenne des deux TE = note du cours).

« Une solution non développée sera considérée comme fausse » : écrire chaque étape, chaque valeur intermédiaire. **Arrondir correctement** les résultats. Pas de téléphone.

Formulaires et tables (éd. 2015) : référence habituelle de l'école — vérifier en cours qu'elle est admise.

À faire confirmer en cours (16.10 ou 30.10)

Portée : le chapitre 2 (zéros, intégrales) est-il au TE1 ? Calculatrice et formulaire admis ? **Coquilles relevées** (contrôlées par le script) : Tab. 1.4, ϵ double imprimé $2{,}204 \cdot 10^{-16}$ au lieu de $2{,}2204 \cdot 10^{-16}$; Tab. 1.2, décalage de la précision étendue 16380 (IEEE : $2^{14} - 1 = 16383$) ; § 2.2.2, Simpson est exacte jusqu'au degré 3 inclus ; série 2, ex. 5, Viète $x_1 + x_2 = -a$.

2. Les entiers sur l'ordinateur (poly § 1.1.1)

Bit (0 ou 1), **octet** = 8 bits (non signé : 0 … 255), **mot** pour le calcul (de 8 à 80 bits selon la machine, en pratique un multiple de l'octet). Pour un entier **signé**, le premier bit donne le signe ; deux codages (Tab. 1.1) :

Codage sur un octet	Étendue	Particularité
signe + valeur absolue (7 bits)	−127 … + 127	deux zéros : +0 = 0 0000000, −0 = 1 0000000
complément à 2	−128 … + 127	un seul zéro ; −1 = 1 1111111, −128 = 1 0000000

Complément à 2 de $-a$ sur N bits : écrire a en binaire, inverser tous les bits, ajouter 1 $\quad (\Leftrightarrow\ 2^N - a)$

Exemple : -5 sur 8 bits : $5 = 00000101 \to 11111010 \to$ **11111011**. Relecture : bit de signe à 1, donc valeur = motif $- 2^8 = 251 - 256 = -5$.

Entier signé (complément à 2)	2 octets	4 octets	8 octets
étendue	$-32\,768 \dots 32\,767$	$-2\,147\,483\,648 \dots \dots 647$	$-9\,223\,372\,036\,854\,775\,808 \dots \dots 807$

Ariane 5, vol 501 (Kourou, juin 1996)
La vitesse horizontale, un flottant de 64 bits, est convertie en **entier signé de 16 bits**. Sa valeur dépasse $32\,767 = 2^{15} - 1$: la conversion échoue, l'exception est traitée par le contrôle de vol comme une valeur valide, la trajectoire part, l'autodestruction se déclenche (37 s après le départ). Illustration (valeur choisie pour l'exemple) : les 16 bits de 40 000 relus en complément à 2 donnent $-25\,536$; MATLAB, lui, **sature** : `int16(40000)` $= 32\,767$.

En MATLAB, les entiers sont stockés comme des réels (double) ; les opérations sont **exactes tant que le résultat est un entier** représentable, c'est-à-dire jusqu'à $2^{53} = 9\,007\,199\,254\,740\,992$ (`flintmax`) : $2^{53} + 1$ n'existe plus. Remarques du poly : ordre des octets *little / big endian*, différences entre machines.

3. Les réels en virgule flottante (poly § 1.1.2, diapos 3–8)

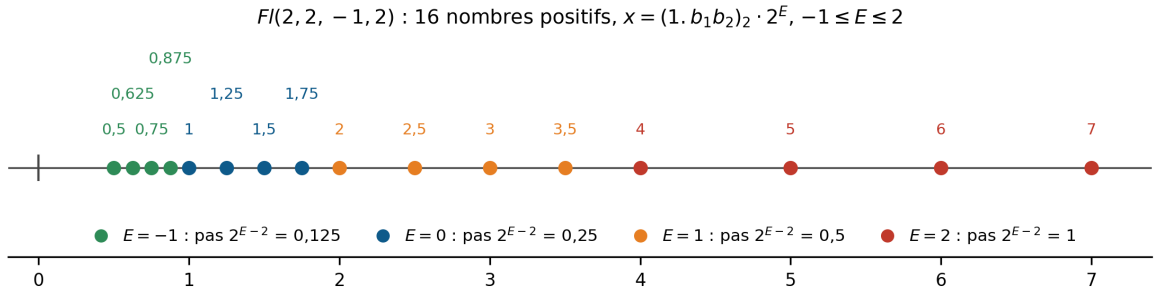
$$x = \pm\,(a_0.b_1b_2\cdots b_n)_\beta \cdot \beta^k, \qquad 1 \le a_0 \le \beta - 1, \quad 0 \le b_i \le \beta - 1, \quad M_1 \le k \le M_2 \qquad \text{Notation : } Fl(\beta, n, M_1, M_2)$$

Normalisé = le premier chiffre a_0 n'est pas nul (en base 2 : $a_0 = 1$) ; zéro a une représentation à part. Exemples du poly : $13{,}524 = 1{,}3524 \cdot 10^1$, $-0{,}0442 = -4{,}42 \cdot 10^{-2}$.

Compter les nombres (question de la diapo 5) : $\beta - 1$ choix pour a_0, β^n pour $b_1 \dots b_n$, $M_2 - M_1 + 1$ exposants (le « +1 » !) :

$$N_+ = (\beta - 1)\,\beta^n\,(M_2 - M_1 + 1) \qquad N_{\text{total}} = 2N_+ + 1 \quad \text{(négatifs et zéro)}$$

Ensemble (diapo 5)	Machine	Positifs N_+	Total $2N_+ + 1$
$Fl(2, 2, -1, 2)$	modèle jouet	16	33
$Fl(2, 3, -2, 2)$	série 2, ex. 1	40	81
$Fl(2, 23, -126, 127)$	IEEE simple	2 130 706 432	$\approx 4{,}26 \cdot 10^9$
$Fl(2, 52, -1022, 1023)$	IEEE double	9 214 364 837 600 034 816	$\approx 1{,}84 \cdot 10^{19}$
$Fl(10, 13, -999, 999)$	TI-85	179 910 000 000 000 000	359 820 000 000 000 001

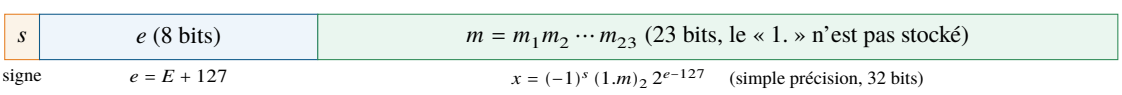


Les nombres ne sont **pas équidistants** : entre 2^E et 2^{E+1} le pas vaut 2^{E-n}, il **double** à chaque exposant. Plus petit positif 0,5, plus grand $(1.11)_2 \cdot 2^2 = 7$.

Deux conventions pour M_1
Le poly écrit $-M_1 \le E \le M_2$; les diapositives et la série 2 écrivent $M_1 \le k \le M_2$ avec M_1 négatif ($Fl(2, 23, -126, 127)$). Au TE, **recopier la convention de l'énoncé** et écrire explicitement l'intervalle des exposants avant de compter.

Le standard IEEE 754 (1985)

$$x = s \cdot (1.m_1m_2\cdots m_n)_2 \cdot 2^E, \qquad E = e - d \quad (e : \text{exposant mémorisé}, d : \text{décalage ou biais})$$



Précision (Tab. 1.2)	longueur	s	m (bits)	d	e (bits)
simple	32	1	23 (+1)	127	8
double (MATLAB)	64	1	52 (+1)	1023	11
étendue	80	1	64	poly : 16380 (IEEE : 16383)	15

« +1 » : le premier bit d'un nombre normalisé vaut toujours 1, on ne le stocke pas (**bit caché**). Le **décalage** évite un bit de signe pour l'exposant : $E = -126 \Leftrightarrow e = 1$. Les valeurs extrêmes de e (tout à 0, tout à 1) sont **réservées** (zéro et dénormalisés ; Inf et NaN).

Nombre	s	e	m
1	0	01111111	000…0
$1+\epsilon$	0	01111111	000…01
−0,5	1	01111110	000…0
1,5	0	01111111	1000…0
2,5	0	10000000	01000…0
realmax	0	11111110	111…1
realmin	0	00000001	000…0
±Inf	0/1	11111111	000…0
NaN	$*$	11111111	non nul

Coder 2,5 (Tab. 1.3) : $2{,}5 = (10.1)_2 = (1.01)_2 \cdot 2^1 \Rightarrow s = 0, E = 1$, $e = 1 + 127 = 128 = 10000000$, $m = 0100\ldots0$.

Décoder : $e = 01111110 = 126 \Rightarrow E = -1$, $m = 0$, $s = 1$: $x = -(1.0)_2 \cdot 2^{-1} = -0{,}5$.

Méthode : (1) signe ; (2) écrire $|x|$ en binaire ; (3) normaliser $(1.\ldots)_2 \cdot 2^E$; (4) $e = E + d$ en binaire sur 8 (11) bits ; (5) m = bits *après* la virgule, complétés par des 0.

Vérification MATLAB : `dec2bin(typecast(single(2.5),'uint32'),32)`.

4. Précision machine, domaine, Inf et NaN

$$\epsilon = 2^{-n} \qquad \text{pas entre } 2^E \text{ et } 2^{E+1} : 2^{E-n} \qquad \text{realmin} = 2^{M_1} \qquad \text{realmax} = (2-\epsilon)\,2^{M_2}$$

ϵ (**précision relative de la machine**) est la distance entre 1 et le nombre machine suivant.

(Tab. 1.4)	simple (binaire)	simple (décimal)	double (binaire)	double (décimal, MATLAB)
ϵ (`eps`)	2^{-23}	$1{,}1921 \cdot 10^{-7}$	2^{-52}	$2{,}2204 \cdot 10^{-16}$
`realmin`	2^{-126}	$1{,}1755 \cdot 10^{-38}$	2^{-1022}	$2{,}2251 \cdot 10^{-308}$
`realmax`	$(2-\epsilon)\,2^{127}$	$3{,}4028 \cdot 10^{38}$	$(2-\epsilon)\,2^{1023}$	$1{,}7977 \cdot 10^{308}$
plus petit dénormalisé ϵ·realmin	2^{-149}	$1{,}4 \cdot 10^{-45}$	2^{-1074}	$4{,}94 \cdot 10^{-324}$

— **Finitude de** m = limite de **précision** ; **finitude de** e = limite du **domaine**.

— ϵ n'est **pas** « le zéro en virgule flottante » : il existe beaucoup de nombres plus petits.

— **Overflow** (> realmax) → `Inf` : $1/\text{Inf} = 0$, $\text{Inf} + \text{Inf} = \text{Inf}$, $1/0 = \text{Inf}$. Indéfini → `NaN` : $0/0$, $\text{Inf} - \text{Inf}$. Exemple de la diapo 14 : $170! \approx 7{,}26 \cdot 10^{306}$ passe, $171!$ et donc $175!$ donnent `Inf`.

— **Underflow** (< realmin) : nombres **dénormalisés** (bit caché à 0) jusqu'à ϵ·realmin, puis 0.

— Complexe = 2 réels (deux fois plus de place) ; matrices stockées **en colonne** en MATLAB et Fortran, en ligne en C.

[Série 2, ex. 4 et 6] Retrouver ϵ, realmin et realmax par programme

```
format long
a = 4/3       % 4/3 n'est pas exact en binaire
b = a - 1
c = 3*b
e = 1 - c     % donne eps = 2^-52, pas 0
```

[Ex. 4] En exact : $a = 4/3$, $b = 1/3$, $c = 1$, $e = 0$. En double, $4/3$ est arrondi : $c = 1 - 2^{-52}$ et $e = 2^{-52} = \epsilon$ (vérifié dans MATLAB : `e == eps` vaut 1). Ce programme servait à trouver ϵ avant le standard. `format long` ne change **que l'affichage**, jamais la précision des calculs.

Boucle (série 2, ex. 6)	Affiche	Compteur	Lecture
$y \leftarrow y/2$ tant que $1 + y \neq 1$	1	$p = 53$	premier incrément perdu : $\epsilon = 2^{-(p-1)} = 2^{-52}$
$x \leftarrow x/2$ tant que $x > 0$	0	$q = 1075$	dernier positif 2^{-1074} (dénormalisé)
$x \leftarrow 2x$ tant que $x \neq \text{Inf}$	Inf	$r = 1024$	2^{1024} déborde ; realmax $= (2-\epsilon)\,2^{1023}$

5. Les erreurs (poly § 1.2, diapos 11–13)

Huit types d'erreurs (diapo 11) : données d'origine (mesure) · programmes · **troncature** · **représentation (arrondi)** · **overflow / underflow** · **opérations** · **annulation** · **souillure**.

$$\text{erreur absolue} = |\tilde{x} - x| \qquad \text{erreur relative} = \frac{|\tilde{x} - x|}{|x|}\ (x \neq 0) \qquad |\tilde{x} - x| \leq \tfrac{1}{2}\,2^{E-n}, \quad \frac{|\tilde{x} - x|}{|x|} \leq 2^{-n} = \epsilon$$

— $\tilde{x}$ = valeur approchée (machine), x = vraie valeur. Les diapositives divisent par $|\tilde{x}|$: $|x-\tilde{x}| \le \epsilon|\tilde{x}|$, soit $(1-\epsilon)\tilde{x} \le x \le (1+\epsilon)\tilde{x}$. L'écart entre les deux définitions est négligeable. **L'erreur relative ne dépend pas de** x : c'est tout l'intérêt du flottant.

— **Chiffres décimaux corrects** (diapo 12) : si $|x-\tilde{x}| \le 0{,}5 \cdot 10^{-t}$, $\tilde{x}$ a t chiffres décimaux corrects. Exemple : $\tilde{x} = 1{,}789$ pour $x = 1{,}794$: $|x-\tilde{x}| = 0{,}005 \le 0{,}5 \cdot 10^{-2} \Rightarrow$ **2** décimales correctes (pas 3), bien que le 9 final « diffère ».

Troncature : la série de Taylor de e^x [poly § 1.2.1, série 2 ex. 7]

On remplace la série infinie par une somme finie :

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} \approx T_N(x) = \sum_{k=0}^{N} \frac{x^k}{k!} \qquad (\text{poly} : N = 15)$$

Récurrence sans puissance ni factorielle : $t_0 = 1$, $t_k = t_{k-1}\, x/k$.

```
function y = mon_exp(x, n)
terme = 1; y = 1;
for k = 1:n
    terme = terme*x/k;
    y = y + terme;
end
```

Erreur relative maximale sur 50 points de $[-1;1]$ (atteinte en $x = -1$) :

n	mesurée (double)	troncature exacte	borne $e/(n+1)!$
4	$1{,}94 \cdot 10^{-2}$	$1{,}94 \cdot 10^{-2}$	$2{,}27 \cdot 10^{-2}$
8	$6{,}80 \cdot 10^{-6}$	$6{,}80 \cdot 10^{-6}$	$7{,}49 \cdot 10^{-6}$
16	$7{,}54 \cdot 10^{-15}$	$7{,}24 \cdot 10^{-15}$	$7{,}64 \cdot 10^{-15}$
20	$4{,}18 \cdot 10^{-16}$ (arrondis)	$5{,}09 \cdot 10^{-20}$	—

Comment choisir N **?** (diapo 11 ; complément : reste de Lagrange) sur $[-1;1]$, erreur relative $\le e/(N+1)!$. Il faut $N = 18$ pour passer sous ϵ ; avec $N = 15$, $e - T_{15}(1) = 5{,}08 \cdot 10^{-14}$.

Troncature contre arrondi

Augmenter n réduit l'erreur de *troncature* (mathématique), pas l'erreur d'*arrondi* (machine). À $n = 20$ la troncature (10^{-20}) est sous ϵ : la courbe d'erreur n'est plus que du « bruit » d'arrondi de l'ordre de 10^{-16}. Inutile d'ajouter des termes.

Étude de cas : le missile Patriot (Dhahran, 25.02.1991)

Horloge en dixièmes de seconde ; temps = tics × 1/10, calculé dans un **registre de 24 bits**.
$1/10 = (0.0001100110011001100110011\ldots)_2$: développement binaire **infini**.
Registre : $(0.00011001100110011001100)_2$ (**troncature**).
Erreur par tic : $0{,}1 - \text{registre} = 0{,}8 \cdot 2^{-23} = 9{,}54 \cdot 10^{-8}$ s (diapo : 0,000000095).

Après **100 h** : $100 \cdot 3600 \cdot 10 = 3{,}6 \cdot 10^6$ tics
$\Rightarrow$ dérive $9{,}54 \cdot 10^{-8} \cdot 3{,}6 \cdot 10^6 =$ **0,343** s.
Scud à 1676 m/s : $0{,}343 \cdot 1676 \approx$ **575** m (diapo : 0,5 km).
Le Scud n'est pas intercepté : 28 morts.

Leçon : une erreur d'arrondi minuscule, *multipliée* par un grand nombre, devient une erreur propagée catastrophique.

6. Opérations, annulation et souillure (poly § 1.2.3, diapos 14–18)

Sur une **machine à 3 chiffres décimaux** dans la mantisse, **chaque résultat intermédiaire est arrondi** :

Propriété perdue	Calcul 1	Calcul 2	Exact
associativité de +	$(99{,}9 + 0{,}54) + 0{,}47 = 100 + 0{,}47 =$ **100**	$99{,}9 + (0{,}54 + 0{,}47) = 99{,}9 + 1{,}01 =$ **101**	100,91
associativité de ×	$(123 \cdot 2{,}56) \cdot 2{,}11 = 315 \cdot 2{,}11 =$ **665**	$123 \cdot (2{,}56 \cdot 2{,}11) = 123 \cdot 5{,}40 =$ **664**	664,3968
$n \cdot a = a + \cdots + a$	$5 \cdot 0{,}444 =$ **2,22**	$0{,}444 + \cdots + 0{,}444 =$ **2,21** (5 termes)	2,22
$a \cdot (1/a) = 1$	$3/3 =$ **1**	$3 \cdot (1/3) = 3 \cdot 0{,}333 =$ **0,999**	1

Règles : additionner en **commençant par les plus petits termes** ; les erreurs d'une addition itérée **s'accumulent** ; préférer x/a à $x \cdot (1/a)$ (le poly, ch. 3 : $7x = 21$ se résout par une division, pas par l'inverse).

Annulation (cancellation) : $a - b$ avec $a \approx b$

[Diapo 17] Machine à 10 chiffres :
$\sqrt{9876} - \sqrt{9875} = 0{,}9937806599 \cdot 10^2 - 0{,}9937303457 \cdot 10^2$
$= 0{,}0000503142 \cdot 10^2 = 0{,}503142???? \cdot 10^{-2}$: 4 chiffres **perdus**.

Remède : réécrire sans soustraction

$$\sqrt{a} - \sqrt{b} = \frac{a-b}{\sqrt{a}+\sqrt{b}}$$

$\dfrac{1}{\sqrt{9876}+\sqrt{9875}} = 0{,}5031418680 \cdot 10^{-2}$ (valeur exacte arrondie ; la machine à 10 chiffres donne $0{,}5031418679 \cdot 10^{-2}$: seul le dernier chiffre bouge).

[Série 2, ex. 5] $x^2 + ax + b = 0$, $a = -10^8$, $b = 1$: $x_{1,2} = -\frac{a}{2} \pm \sqrt{(\frac{a}{2})^2 - b}$.
(a) directe : petite racine $7{,}4506 \cdot 10^{-9}$ — **25,5 % d'erreur** (annulation de deux nombres voisins de $5 \cdot 10^7$) ; produit des racines 0,745 au lieu de 1.
(b) **Viète** $x_1 x_2 = b$: $x_2 = b/x_1 = 1{,}0 \cdot 10^{-8}$, erreur $< \epsilon$, comme `roots([1 -100000000 1])`.

Réflexe : calculer d'abord la racine où $-\frac{a}{2}$ et la racine carrée s'*additionnent* (pas d'annulation), puis l'autre par $x_1 x_2 = b$. (Coquille de l'énoncé : $x_1 + x_2 = -a$, pas a.)

Souillure (smearing, diapo 18) : soustraire des nombres d'ordres de grandeur très différents. Machine à 5 chiffres : $0{,}47211 - 0{,}21468 \cdot 10^{-3} = 0{,}47189532 \rightarrow$ **0,47190** : les derniers chiffres du petit terme sont perdus. Dans une longue somme, ces pertes

s'additionnent ⇒ **sommer du plus petit au plus grand** (en valeur absolue).

```
somme = 0;                % serie 2, ex. 8
for i = 1:10
    somme = somme + 0.1;
end
if abs(somme - 1) <= 1e-5   % pas somme == 1
    disp('Le compte est bon')
else
    disp('Detournement de fonds!')
end
```

0,1 n'est pas représentable : le double stocké vaut 0,1000000000000000055… Après dix additions, `somme` = 0,99999999999999989 (= $1 - 2^{-53}$) : avec `somme == 1`, le programme affiche « Détournement de fonds ! ». De même `0.1 + 0.2 == 0.3` est faux (0,30000000000000004).

Réflexe : jamais de `==` entre flottants calculés ; tester `abs(a - b) <= tol`, avec une tolérance adaptée à l'échelle des nombres.

Condition d'un problème (poly § 1.2.4) contre stabilité d'une méthode

Un problème **bien posé** a une solution qui existe, est unique et dépend continûment des données. Il peut quand même être **mal conditionné** : la condition est une propriété *du problème*, indépendante de la méthode. La petite racine de la série 2 n'est *pas* un problème mal conditionné : c'est la *formule* (a) qui est instable, et la formule (b) la calcule à ϵ près.

7. MATLAB pour le TE (série 1, introduction à MATLAB)

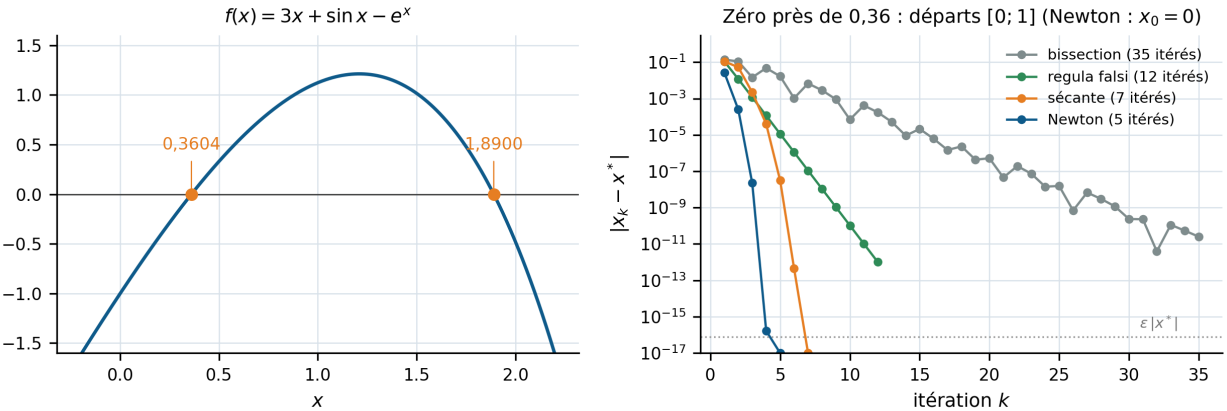
Commande	Effet	Exemple (série 1, vérifié R2025a)
`format long` / `short`	affichage seulement	`pi` → 3.141592653589793
`format rat`, `rats`	approximation rationnelle affichée	$\pi \to 355/113$, $e \to 1457/536$
`x = 1:10`, `x'`, `x(:)`	vecteur ligne ; transposée ; `x(:)` rend *toujours* une colonne	ex. 4
`*` contre `.*`, `./`, `.^`	produit matriciel contre élément par élément	`x*y'` = 5052 (ex. 5)
`cumprod`, `sum`, `prod`	produits cumulés, sommes (par colonne !)	`cumprod(1:15)` : 15! = 1 307 674 368 000
`magic(n)`, `sum(T)`, `sum(T')`	colonnes, puis lignes	constante $(n^3+n)/2$ = 505, 4010, 13 515
`rem(a,2)`	reste de la division	`pair` : tous les éléments pairs ? (ex. 8)
`linspace(a,b,n)`	n points, bornes **incluses**	`linspace(-1,1,50)` (série 2)
`f = @(x) …`	fonction anonyme	`f(7)` = −1074,976 (poly § 2.1.5)
`plot`, `hold on`, `subplot`	tracés, superposition, grille de graphes	`xlabel`, `title`, `grid`, `legend`
`eps`, `realmin`, `realmax`, `Inf`, `NaN`	constantes machine (double)	`flintmax` = 2^{53}

```
function t = pair(a)          % ex. 8
t = all(rem(a, 2) == 0);      % 1 si tous pairs
% prof : t = prod(rem(a,2) - 1) ~= 0;
```

[Ex. 3] $e^{\pi} = 23{,}1407 > \pi^{e} = 22{,}4592$. **[Ex. 10]** $x = y^2$ n'est pas une fonction de x : tracer les deux branches $\pm\sqrt{x}$ (le prof utilise `sqrt(abs(x))`, ce qui ajoute des branches parasites pour $x < 0$). **Au TE** : savoir lire un petit programme (boucle `for`/`while`, `if`) et donner sa sortie, comme en série 2, ex. 6.

8. Recherche de zéro (poly § 2.1) — probable au TE

Exemple du poly : $f(x) = 3x + \sin x - e^x$, deux zéros (graphiquement ≈ 0,35 et ≈ 1,9) ; on cherche le premier. $f(0) = -1$, $f(1) = 1{,}1232$, $f(2) = -0{,}4799$; $f'(x) = 3 + \cos x - e^x$.



Zéros $x^* = 0{,}360422$ et $1{,}890030$. Itérés jusqu'au critère du poly avec `tol` = 10^{-10} : bissection 35, regula falsi 12, sécante 7, Newton 5.

Méthode	Itération	+	−
bissection	$x = \frac{x_0+x_1}{2}$; garder la moitié où f change de signe	convergence garantie	lente ; 2 départs
sécante	$x_2 = x_1 - f(x_1)\dfrac{x_1-x_0}{f(x_1)-f(x_0)}$; puis $x_0 \leftarrow x_1, x_1 \leftarrow x_2$	rapide	2 départs ; peut diverger
regula falsi	même formule, mais on garde l'**encadrement** (bissection + sécante)	garantie ; plus rapide que bissection (normalement)	peut être lente
Newton	$x_1 = x_0 - \dfrac{f(x_0)}{f'(x_0)}$ (tangente)	très rapide ; 1 départ ; plusieurs variables	f' requise ; divergence, cycles

Bissection : erreur après n itérations $< \dfrac{|x_1-x_0|}{2^n} \quad\Rightarrow\quad n \geq \log_2 \dfrac{|x_1-x_0|}{\text{tol}}$ $\qquad$ ($[0;1]$, 10^{-6} : $n = 20$)

Bissection sur $[0;1]$ (si $f(x)f(x_0) < 0 : x_1 \leftarrow x$, sinon $x_0 \leftarrow x$)

k	$[x_0;x_1]$	milieu x	$f(x)$
1	$[0;1]$	0,5	+0,3307
2	$[0;0{,}5]$	0,25	−0,2866
3	$[0{,}25;0{,}5]$	0,375	+0,0363
4	$[0{,}25;0{,}375]$	0,3125	−0,1219
5	$[0{,}3125;0{,}375]$	0,34375	−0,0420

Newton depuis $x_0 = 0 : f(0) = -1, f'(0) = 3$
$x_1 = 0 - \frac{-1}{3} = 0{,}333333$;
$f(x_1) = -0{,}068418, f'(x_1) = 2{,}549346$
$x_2 = 0{,}333333 + \frac{0{,}068418}{2{,}549346} = 0{,}360171$;
$x_3 = 0{,}360422$ (erreur $< 10^{-7}$), x_4 : erreur $< 10^{-13}$. Le nombre de décimales justes **double** à chaque pas (convergence quadratique).

Sécante ($x_0 = 0$, $x_1 = 1$) : 0,470990 ; 0,307508 ; 0,362613 ; 0,360461…
Regula falsi : 0,470990 ; 0,372277 ; 0,361598… le bord $x_0 = 0$ ne bouge jamais.

MATLAB : `fzero` mélange bissection (lent), sécante (moyen) et IQI, interpolation quadratique inverse (rapide). Depuis le seul point $x = 1$, R2025a renvoie le zéro **de gauche** (0,3604), pas celui de 1,89 : pour viser un zéro, donner un **intervalle** où f change de signe. Systèmes $f(x,y) = 0$, $g(x,y) = 0$: `fsolve`.

Critère d'arrêt du poly : écart *relatif* entre deux itérés, $|x_1 - x_0| > \text{tol} \cdot |x_1|$.

```
f  = @(x) 3*x + sin(x) - exp(x);
x0 = 0; x1 = 1; tol = 1e-10; k = 0;
while abs(x1 - x0) > tol*abs(x1)
    x = (x0 + x1)/2;
    if f(x)*f(x0) < 0, x1 = x; else, x0 = x; end
    k = k + 1;
end
fzero(f, 1)        % 0.360421702960324
fzero(f, [1 2])    % 1.890029729251985
```

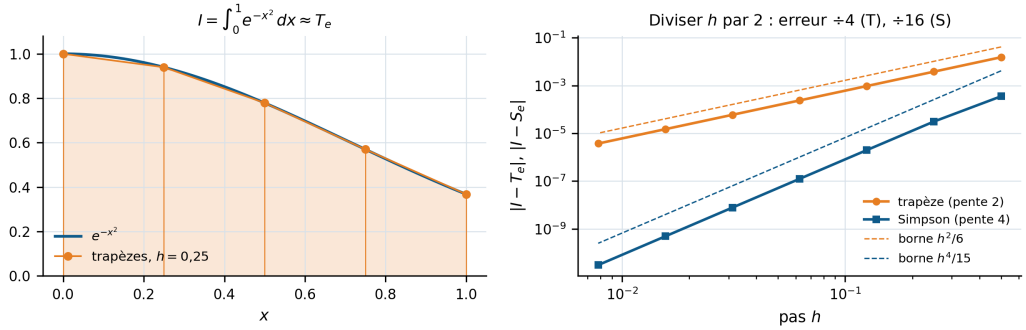
Pièges de la recherche de zéro
Bissection et regula falsi exigent $f(x_0)f(x_1) < 0$ *au départ* ; la sécante et Newton non (d'où le risque de divergence). Newton près d'un extremum ($f' \approx 0$) envoie l'itéré très loin. Regula falsi garde souvent un bord fixe : avec le critère du poly $|x_2 - x_1|$, vérifier que c'est bien x_1 qui bouge (ici $x_0 = 0$ reste fixe et tout va bien).

9. Intégration numérique (poly § 2.2) — probable au TE

$I = \int_a^b f(x)\,dx \approx \sum_i w_i f(x_i)$ (les w_i sont les **poids**). Pas constant h, n points $x_1, \ldots, x_n$:

$$T_e = h\Big[\tfrac{1}{2}f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + \tfrac{1}{2}f(x_n)\Big] \qquad |I - T_e| \leq \frac{x_n - x_1}{12}\,h^2 \max_{[x_1,x_n]} |f''|$$

$$S_e = \frac{h}{3}\Big[f(x_1) + 4f(x_2) + 2f(x_3) + 4f(x_4) + \cdots + 4f(x_{n-1}) + f(x_n)\Big] \ (n \text{ impair}) \qquad |I - S_e| \leq \frac{x_n - x_1}{180}\,h^4 \max_{[x_1,x_n]} |f^{(4)}|$$



Trapèzes sous e^{-x^2} ($h = 0{,}25$) ; erreurs mesurées et bornes du poly : pente 2 (trapèze), pente 4 (Simpson).

Exemple du poly : $I = \int_0^1 e^{-x^2}dx = 0{,}746824$, $h = 0{,}25$ ($n = 5$) : f : 1 ; 0,939413 ; 0,778801 ; 0,569783 ; 0,367879.

$T_e = 0{,}25\,[0{,}5 + 0{,}939413 + 0{,}778801 + 0{,}569783 + 0{,}183940]$ $= \mathbf{0{,}742984}$ (erreur 0,00384).

$S_e = \frac{0{,}25}{3}\,[1 + 4(0{,}939413 + 0{,}569783) + 2 \cdot 0{,}778801 + 0{,}367879]$ $= \mathbf{0{,}746855}$ (erreur $3{,}1 \cdot 10^{-5}$).

Bornes : $\max|f''| = 2$, $\max|f^{(4)}| = 12$ (en $x = 0$) $\Rightarrow h^2/6 = 0{,}0104$ et $h^4/15 = 2{,}6 \cdot 10^{-4}$: respectées.

$h = 0{,}5$: $T_e = 0{,}731370$, $S_e = 0{,}747180$. Diviser h par 2 : erreur ÷4,0 (T), ÷11,4 (S, puis ÷15,8 de 0,25 à 0,125 : on tend vers 16).

```
x = 0:0.25:1; h = 0.25;
f = exp(-x.^2); n = length(f);
T = trapz(x, f)                  % 0.742984
S = h/3*(f(1) + 4*sum(f(2:2:n-1)) ...
    + 2*sum(f(3:2:n-2)) + f(n))  % 0.746855
```

Exactitude : trapèze jusqu'au degré 1, Simpson jusqu'au degré 3

Trapèze exact si $f'' = 0$ (droites) ; Simpson exact si $f^{(4)} = 0$, donc pour les polynômes de degré ≤ 3 (le poly écrit « degré plus petit que 3 » : x^3 est pourtant intégré exactement). Exemple : $\int_0^2 x^3dx = 4$; $h = 1$: $T_e = 5$, $S_e = \frac{1}{3}(0 + 4 + 8) = 4$. Simpson exige un nombre **impair** de points. MATLAB : `trapz`, et `quad` (Simpson adaptative ; `integral` dans les versions récentes).

10. Méthode au TE — quelle question, quel réflexe

On te demande…	Réflexe
un entier négatif en binaire	complément à 2 : binaire de $\lvert a\rvert$, inverser, +1 ; vérifier en relisant (motif -2^N)
de coder / décoder un flottant IEEE	normaliser $(1.\ldots)_2 \cdot 2^E$, $e = E + d$, m = bits après la virgule ; écrire $s\lvert e\rvert m$
de compter les nombres de $Fl(\beta, n, M_1, M_2)$	$(\beta - 1)\beta^n(M_2 - M_1 + 1)$ positifs, ×2 + 1 avec négatifs et zéro
ϵ, realmin, realmax, le pas	2^{-n}, 2^{M_1}, $(2 - \epsilon)2^{M_2}$, pas 2^{E-n} ; dessiner la droite réelle
un calcul sur une machine à t chiffres	arrondir **après chaque opération** ; comparer avec l'ordre inverse et la valeur exacte
pourquoi un résultat est faux	nommer le type d'erreur (8 types) : annulation, souillure, overflow, troncature…
de corriger une formule instable	rationaliser ($\sqrt{a} - \sqrt{b}$), utiliser Viète ($x_2 = b/x_1$), sommer du plus petit au plus grand
la sortie d'un programme MATLAB	dérouler la boucle à la main, compter les passages ; penser ϵ, realmin, Inf
un zéro par bissection / Newton	tableau des itérés (k, intervalle, x, $f(x)$) ; $n \geq \log_2(\lvert x_1 - x_0\rvert/\text{tol})$
une intégrale par trapèze / Simpson	tableau des $f(x_i)$, poids $\frac{1}{2}, 1, \ldots$ ou $1, 4, 2, \ldots, 4, 1$; borne d'erreur avec $\max\lvert f''\rvert$ ou $\max\lvert f^{(4)}\rvert$

11. Mémo des pièges

1. Complément à 2 : $-128 \ldots 127$ (un seul zéro) ; signe + valeur absolue : $-127 \ldots 127$ (deux zéros).
2. Compter les exposants : $M_2 - M_1 + 1$, pas $M_2 - M_1$.
3. IEEE : la mantisse stockée n'inclut *pas* le 1 caché ; $e = E + 127$ (simple), $E + 1023$ (double).
4. $\epsilon = 2^{-52} \approx 2{,}2204 \cdot 10^{-16}$ (le Tab. 1.4 du poly imprime 2,204).
5. ϵ n'est pas le plus petit nombre ; realmin non plus (dénormalisés).
6. Le pas entre flottants double à chaque puissance de 2.
7. `format long` ne change pas la précision, seulement l'affichage.
8. Jamais `==` entre flottants calculés : tolérance.
9. Machine à t chiffres : arrondir après *chaque* opération.
10. L'ordre des opérations change le résultat : petits termes d'abord.
11. Annulation : soustraction de nombres voisins ; souillure : de nombres d'ordres de grandeur différents.
12. Plus de termes de Taylor ne bat pas l'arrondi.
13. MATLAB sature : `int16(40000)` = 32 767.
14. Bissection : vérifier $f(x_0)f(x_1) < 0$ avant de commencer.
15. Newton : écrire f' correctement ; éviter $f' \approx 0$.
16. Simpson : nombre de points impair ; poids $1, 4, 2, \ldots, 2, 4, 1$.

12. Auto-test (réponses en bas de page)

1. Dans $Fl(2, 3, -1, 1)$: nombre de positifs, plus petit, plus grand, et ϵ ?
2. Coder $-6{,}5$ en simple précision IEEE (s, e, m).
3. Quel réel code le mot simple précision 0 10000010 0110 … 0 ?
4. Écrire -37 sur 8 bits en complément à 2.
5. Machine à 4 chiffres : calculer $\sqrt{1001} - \sqrt{1000}$ directement, puis par la forme rationalisée. Combien de décimales correctes dans chaque cas ?
6. Pourquoi `0.1 + 0.2 == 0.3` est-il faux en MATLAB ? Comment écrire le test ?
7. Patriot : quelle dérive d'horloge et quel écart (Scud à 1676 m/s) après 8 h de fonctionnement ?
8. Bissection de $f(x) = 3x + \sin x - e^x$ sur $[1; 2]$: deux premiers milieux, et nombre d'itérations pour une erreur $< 10^{-4}$.
9. Même f : un pas de Newton depuis $x_0 = 2$.
10. $\int_0^2 x^3 dx$ par le trapèze et par Simpson avec $h = 1$. Commenter.

Réponses. 1) $(2-1) \cdot 2^3 \cdot (1-(-1)+1) = 24$ positifs ; min $(1.000)_2 \cdot 2^{-1} = 0{,}5$; max $(1.111)_2 \cdot 2^1 = 3{,}75$; $\epsilon = 2^{-3} = 0{,}125$. 2) $6{,}5 = (110.1)_2 = (1.101)_2 \cdot 2^2$: $s = 1$, $e = 129 = 10000001$, $m = 1010\ldots0$. 3) $e = 130$ donc $E = 3$; $(1.011)_2 = 1{,}375$; $x = 1{,}375 \cdot 8 = 11$. 4) $37 = 00100101$, inversé 11011010, +1 : 11011011. 5) $\sqrt{1001} \to 31{,}64$, $\sqrt{1000} \to 31{,}62$: différence 0,02, seulement 2 décimales correctes (exact 0,0158074) ; $1/(31{,}64 + 31{,}62) = 1/63{,}26 = 0{,}01581$: 5 décimales correctes. 6) 0,1 et 0,2 ne sont pas représentables exactement ; la somme vaut 0,30000000000000004. Tester `abs(a - b) <= tol`. 7) $8 \cdot 3600 \cdot 10 = 288\,000$ tics $\times\, 9{,}54 \cdot 10^{-8}$ s $= 0{,}0275$ s ; $0{,}0275 \cdot 1676 \approx 46$ m. 8) $f(1{,}5) = 1{,}0158 > 0$ et $f(2) < 0$: intervalle $[1{,}5; 2]$, puis $x = 1{,}75$, $f(1{,}75) = 0{,}4797 > 0$: $[1{,}75; 2]$; $n \geq \log_2(10^4) = 13{,}3$, donc 14 itérations. 9) $f(2) = -0{,}4799$, $f'(2) = 3 + \cos 2 - e^2 = -4{,}8052$: $x_1 = 2 - \frac{-0{,}4799}{-4{,}8052} = 1{,}900158$. 10) $T_e = 1 \cdot (0/2 + 1 + 8/2) = 5$; $S_e = \frac{1}{3}(0 + 4 \cdot 1 + 8) = 4$, valeur exacte : Simpson est exacte jusqu'au degré 3.